

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: Deep Learning

COURSE CODE: MLA0403

FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality.
(BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I **Uppu Sye Kumar/192325121** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:U.Sye Kumar

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

- Identify the conceptual error in the likelihood formulation.
- Explain how this mistake affects parameter estimation.
- Suggest the correct mathematical formulation and reasoning.
- How would using log-likelihood fix computational issues?

a) Identify the conceptual error in the likelihood formulation.

- The likelihood function should be the **product of probabilities** of all independent observations.
- The student incorrectly used the **sum of probabilities**, which does not represent the joint probability of the dataset.

Incorrect:

$$L(\theta) = \sum_{i=1}^n P(x_i | \theta)$$

Correct:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

b) Explain how this mistake affects parameter estimation.

- The model no longer maximizes the probability of observing the entire dataset.
- Parameter estimates become incorrect.
- The estimator becomes statistically inconsistent.
- Accuracy remains poor even with large datasets.

c) Suggest the correct mathematical formulation and reasoning.

For independent samples:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

The optimal parameter is

$$\hat{\theta} = \arg \max_{\theta} L(\theta)$$

This formulation correctly represents the probability of observing all training samples simultaneously.

d) How would using log-likelihood fix computational issues?

Log-likelihood:

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

Advantages:

- Converts products into sums.
- Prevents numerical underflow.
- Simplifies differentiation.
- Makes optimization faster and more stable.

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

- a) List possible implementation errors in the learning algorithm.
- b) Analyze how incorrect learning rate or weight updates affect convergence.
- c) Identify dataset-related issues that may cause this problem.
- d) Propose modifications to ensure convergence.

a) List possible implementation errors in the learning algorithm.

- Incorrect weight update rule.
- Wrong bias update.
- Missing activation function.
- Incorrect prediction threshold.
- Wrong label encoding (using 0/1 instead of $-1/+1$).
- Incorrect stopping condition.
- Data not shuffled.

b) Analyze how incorrect learning rate or weight updates affect convergence.

- Very high learning rate $\rightarrow$ weights oscillate and never converge.
- Very low learning rate $\rightarrow$ learning becomes extremely slow.
- Incorrect update direction increases classification error.

Correct update rule:

$$w = w + \eta(y - \hat{y})x$$

where:

- w =weights
- η =learning rate
- y =actual output
- $\hat{y}$ =predicted output

c) Identify dataset-related issues that may cause this problem.

- Dataset not actually linearly separable.
- No feature normalization.
- Incorrect labels.

- Duplicate or noisy samples.
- Class imbalance.

d) Propose modifications to ensure convergence.

- Normalize features.
- Use proper learning rate.
- Shuffle data every epoch.
- Correct weight update.
- Verify label encoding.
- Remove noisy samples.
- Use Pocket Algorithm if data is not perfectly separable.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

- Identify possible mathematical or coding errors in gradient computation.
- Explain the role of activation functions in gradient flow.
- Analyze how vanishing or exploding gradients affect training.
- Suggest techniques to fix the issue (e.g., normalization, initialization).

a) Identify possible mathematical or coding errors in gradient computation.

- Wrong derivative calculation.
- Incorrect chain rule implementation.
- Gradient sign error.
- Missing bias gradient.
- Wrong matrix dimensions.
- Incorrect learning rate.

b) Explain the role of activation functions in gradient flow.

Activation functions introduce non-linearity.

Examples:

- Sigmoid
- Tanh
- ReLU

Poor activation choices may reduce gradients, slowing learning.

c) Analyze how vanishing or exploding gradients affect training.

Vanishing Gradient

- Gradients become extremely small.
- Early layers stop learning.
- Loss decreases very slowly.

Exploding Gradient

- Gradients become extremely large.
- Weight updates become unstable.
- Loss increases rapidly or becomes NaN.

d) Suggest techniques to fix the issue.

- Xavier Initialization

- He Initialization
- Batch Normalization
- Gradient Clipping
- ReLU activation
- Proper learning rate
- Adam optimizer

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

- Explain why batch gradient descent is slow in large datasets.
- Analyze the cause of fluctuations in SGD.
- Compare the error behavior of Mini-Batch Gradient Descent.
- Recommend the best approach and justify with error trade-offs.

a) Explain why Batch Gradient Descent is slow in large datasets.

Batch Gradient Descent computes gradients using **all training samples** before every update. Therefore:

- High computational cost
- Large memory usage
- Slow parameter updates

b) Analyze the cause of fluctuations in SGD.

SGD updates weights after each training example.

Advantages:

- Faster updates
- Escapes local minima

Disadvantages:

- High variance
- Noisy loss curve
- Oscillations around optimum

c) Compare the error behavior of Mini-Batch Gradient Descent.

Method	Error Behavior
Batch GD	Smooth but slow convergence
SGD	Fast but noisy convergence
Mini-Batch GD	Balanced, stable, and efficient convergence

Mini-Batch Gradient Descent reduces variance while maintaining faster training.

d) Recommend the best approach and justify with error trade-offs.

Recommendation: Mini-Batch Gradient Descent

Reasons:

- Faster than Batch GD.
- More stable than SGD.

- Efficient GPU utilization.
- Better convergence.
- Lower computational cost.

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

- Explain how high dimensionality increases model error.
- Identify signs of overfitting in this scenario.
- Analyze why distance-based learning fails in high dimensions.

a) Explain how high dimensionality increases model error.

As the number of features increases:

- Data becomes sparse.
- Distance between points becomes less meaningful.
- More training samples are required.
- Model memorizes training data, leading to overfitting.

b) Identify signs of overfitting in this scenario.

- Very high training accuracy.
- Low testing accuracy.
- Large gap between training and testing performance.
- Poor generalization.
- High variance.

c) Analyze why distance-based learning fails in high dimensions.

Algorithms such as KNN and K-Means rely on distances.

In high dimensions:

- Distances between points become nearly equal.
- Nearest neighbors are less distinguishable.
- Similarity measures lose effectiveness.
- Classification accuracy decreases.

d) Suggest dimensionality reduction or regularization techniques to improve performance.

Dimensionality Reduction

- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- Feature Selection
- Autoencoders
- t-SNE (for visualization)

Regularization

- L1 (Lasso)
- L2 (Ridge)

- Elastic Net
- Dropout (for neural networks)
- Early Stopping

Benefits

- Reduces overfitting.
- Improves generalization.
- Speeds up training.
- Enhances prediction accuracy.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem Solving	20	Logical, well structured, and	Mostly logical approach with	Basic approach with some	Illogical or unclear

Approach		efficient approach	minor gaps	inconsistencies	approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand